Development Plan — [Project name]

Field	Entry
Project	[Project name]
Document	Development Plan
Version	[1.0]
Based on	BRD v[x.x], Gap Analysis v[x.x], Spike v[x.x]
Date	[YYYY-MM-DD]
Author	[Lead developer]
Status	Draft / Active / Complete

Workflow step: 4. Development. **Next step:** 5. Testing (QA).

1. Summary

[Version] is built in [n] sprints of [n] weeks, from [start] to [end], by [n] developers.

2. Entry criteria

- ☐ Spike result is **Go**
- ☐ BRD approved, including any spike changes
- ☐ Technology and licence choices fixed
- ☐ Team assigned
- ☐ Repository, issue tracker and CI ready

3. Architecture

Guidance: add a diagram if the system has more than a few parts. List every module with the requirements it serves.

Layer	Module	Responsibility	Requirements
[UI / API / Service / Data]	[module]	[What it does]	[BR-xx, NFR-xx]
[ ]	[ ]	[ ]	[ ]

4. Tech stack and project structure

Area	Choice
Language	[ ]
Framework / UI	[ ]
Data / storage	[ ]
Tests	[ ]
Packaging / hosting	[ ]
CI	[ ]

```
[project]/
├── src/
├── tests/
├── docs/
└── [config files]
```

5. Module interfaces and data model

Guidance: define the contract between modules (inputs, outputs, errors) and the main data structures, so modules can be built and tested separately.

Module	Input	Output	Errors
[ ]	[ ]	[ ]	[ ]

6. Configuration

Key	Default	Purpose
[setting]	[value]	[BR-xx]

7. Error handling and fallbacks

Situation	Behaviour	Requirement
[Failure case]	[What the system does]	[NFR-xx]

8. Logging, security and privacy

- [What is logged, where, for how long]
- [What is never logged]
- [Authentication, secrets handling, network rules]

9. Sprint plan

Sprint	Dates	Goal	Requirements	Demo at sprint end
1	[ ]	Foundation	[ ]	[ ]
2	[ ]	[ ]	[ ]	[ ]
3	[ ]	[ ]	[ ]	[ ]
[n]	[ ]	Release readiness	[ ]	Build handed to QA

Change capacity: each sprint keeps [20]% of time for approved change requests. A [1]-week fix sprint follows UAT.

10. Engineering practices

Branching: `main` always releasable; `develop` for finished work; `feature/BR-xx-name`, `fix/123-name`, `cr/CR-xxx-name`.

Code standards: [formatter, linter, type checks, review rules].

Testing during development

Test type	Covers	When
Unit	[ ]	Every pull request
Integration	[ ]	Every pull request / nightly
UI / end-to-end	[ ]	[ ]

11. Handling change requests

CR class	What development does
A. Minor	Fix in current sprint, inside change capacity
B. Requirement tweak	Update logic, config and tests after BRD update
C. New technical approach	Build after a passed re-spike; keep the old option switchable until QA passes
D. New feature	Plan into a sprint as new work
E. Fallback	Switch on the agreed fallback; update release notes

Rules: CR ID in branch, pull request and release notes; each CR adds or updates a test; work over [50]% of the approved effort returns to the change board.

12. Developer setup

1. [Install prerequisites]
2. [Clone and install dependencies]
3. [Configure secrets / access]
4. [Run the app]
5. [Run tests]

13. CI, packaging and release

Stage	Runs on	Does
Lint and tests	Every pull request	[ ]
Nightly	[ ]	[ ]
Release build	Tag on `main`	[ ]

14. Roles and risks

Role	Name	Responsible for
Product owner	[ ]	Priorities, sprint demos, change decisions
Lead developer	[ ]	Architecture, reviews, release builds
Developer	[ ]	[ ]
QA	[ ]	Test plan, bug triage

Risk	Mitigation
[ ]	[ ]

15. Definition of done and handoff to QA

A feature is done when:

- ☐ Merged after review, tests passing
- ☐ Meets the BRD acceptance condition
- ☐ Documented (user-facing text, release notes)
- ☐ For CR work: CR linked, documents updated

Handoff to QA when:

- ☐ All Must requirements done
- ☐ Release candidate built from `main`
- ☐ Release notes and known issues written
- ☐ Open bugs: none critical, at most [n] major

16. Requirement traceability

Requirement	Priority	Module(s)	Sprint	Verified by
BR-01	[ ]	[ ]	[ ]	[Test type / case]

17. Change log

Version	Date	Change
1.0	[YYYY-MM-DD]	First version